

PROJECT REPORT

Simple Grocery Billing System

A Python console application for itemised billing with coupons and tax

Submitted by	
Roll / Enrolment No.	
Course / Programme	
Guide / Instructor	
Institution	
GitHub Repository	
Date of Submission	30 September 2026

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Small retail stores still depend heavily on manual calculation at the billing counter. Adding up prices, applying discounts and calculating tax by hand is slow and easy to get wrong. The **Simple Grocery Billing System** is a console-based Python program that automates this process. A customer chooses items from a fixed catalogue, enters quantities, applies optional coupon codes and receives an itemised bill with a 5% tax.

The store sells ten products (bread, butter, cheese, gum, chips, chicken, ice cream, toothpaste, chocolate and milk). Chicken is priced per kilogram and milk per litre, while the other items are priced per piece. The system also includes a small promotional feature: a customer who enters a valid 10-digit phone number receives a giveaway coupon code that can be used on the same bill.

The project started as a single script of about 100 lines that used NumPy arrays and contained a number of logic errors (described in Section 12). The final version is a restructured, validated and tested program in plain Python with a clear separation between the user interface, the business logic and the data.

3. Problem Statement

To design and implement a simple grocery billing system that lets a user select products from a store catalogue, specify quantities, apply item-specific and bill-level discount coupons, and produce an accurate, itemised final bill including tax, while handling invalid input gracefully.

Scope. The system is a single-user, command-line program with in-memory data. It covers billing only. It does not cover stock management, payments, user accounts or receipt printing hardware.

Objectives

- Display the catalogue with prices and units.
- Accept multiple items and quantities in one entry, and allow more items to be added afterwards.
- Support item-specific coupons (RISEUP, CLOCKADOODLEDO, SMOOTHLIKE) and bill-level coupons (MONEY, MONEYHEIST, FATHER).
- Give a promotional coupon after a valid phone number is entered.
- Compute subtotal, 5% tax and grand total correctly.
- Reject invalid input with clear messages instead of crashing.

4. Functional Requirements

ID	Requirement
FR-1	The system shall display all available items with their prices and units (each / per kg / per litre).
FR-2	The system shall accept a space-separated list of item names and a matching list of quantities.
FR-3	The system shall reject unknown item names, non-numeric or non-positive quantities, and mismatched item/quantity counts, and ask the user to re-enter.
FR-4	The system shall let the user add more items after the first entry; repeated items shall be merged by adding quantities.
FR-5	The system shall accept a number N of item-specific coupon codes and then N codes. Valid codes reduce the unit price of one specific item: RISEUP = 50% off butter; CLOCKADOODLEDO = one-third off chicken; SMOOTHLIKE = one-third off cheese.
FR-6	The system shall ask for a phone number. If it has exactly 10 digits, the coupon code MONEYHEIST is shown; otherwise a message says no coupon is given.
FR-7	The system shall accept bill-level coupon codes: MONEY = 30% off, MONEYHEIST = 20% off, FATHER = 40% off. Invalid codes shall be ignored with a message.
FR-8	The system shall compute line totals, the subtotal after discounts, 5% tax and the grand total.
FR-9	The system shall print an itemised bill showing item, quantity, rate and amount, followed by the totals.

5. Non-functional Requirements

Attribute	Requirement
Usability	Clear prompts, numbered menu and readable bill layout; error messages explain what to fix.
Reliability	No crash on bad input (unknown items, letters instead of numbers, empty entries, mismatched counts).
Correctness	Totals must match hand calculation. Verified with automated unit tests.
Maintainability	Prices and coupons are stored in dictionaries at the top of the file, so changing them does not need logic changes. Functions are small and single-purpose.
Testability	Pricing logic is separated from input/output so it can be tested without typing input.
Portability	Runs on any system with Python 3.8 or later; no third-party libraries.
Performance	Response is instantaneous for any realistic cart size.
Security / Privacy	The phone number is only length- and digit-checked and is never stored or written to disk.

6. System Architecture

The program follows a simple three-layer structure. This is a common pattern even for small programs, because it keeps the code that talks to the user separate from the rules that calculate money.

- **Presentation layer** - console functions that print prompts and read input (`show_menu`, `ask_cart`, `ask_extra_items`, `ask_item_coupons`, `offer_giveaway_coupon`, `ask_bill_coupons`, `print_bill`).
- **Business logic layer** - `parse_cart()` validates and merges input; `calculate_bill()` applies item coupons, computes line totals, applies bill coupons and adds tax.
- **Data layer** - Python dictionaries `ITEMS`, `ITEM_COUPONS`, `BILL_COUPONS` and the constant `TAX_RATE`. Data is in memory; no database or file is used.

`main()` acts as the controller that calls the UI functions in order and passes the results to the business logic.

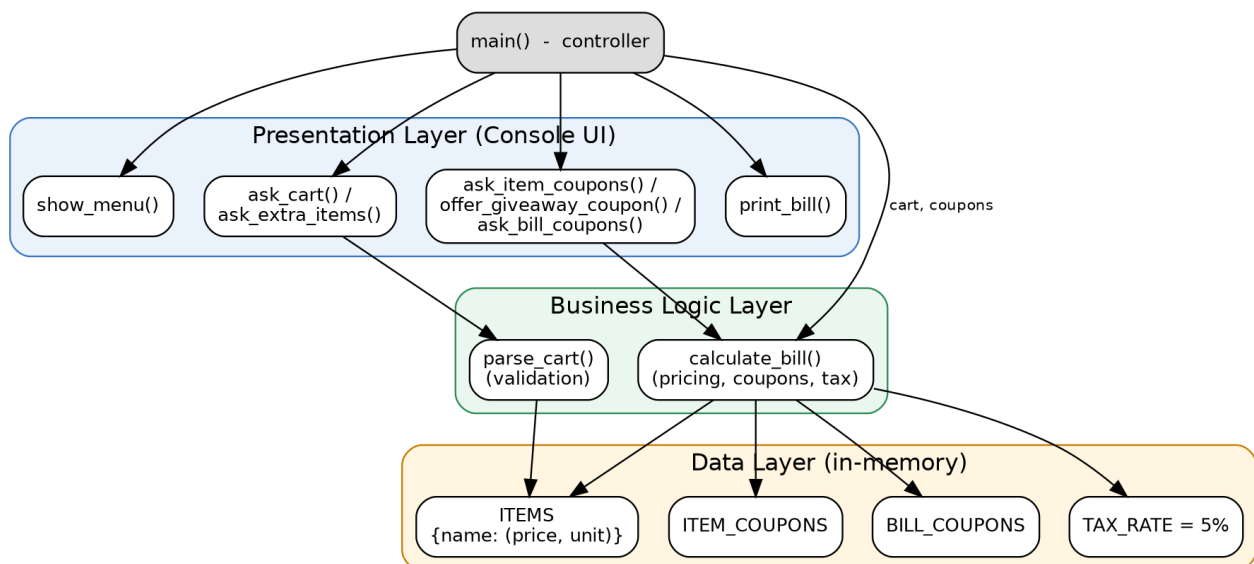


Figure 1: Layered architecture of the grocery billing system

7. Design Diagrams

7.1 Use Case Diagram

A single actor, the customer, interacts with the system. Input validation is included by the item-selection use cases, and the giveaway coupon extends the bill-coupon step because its code (MONEYHEIST) can then be used there.

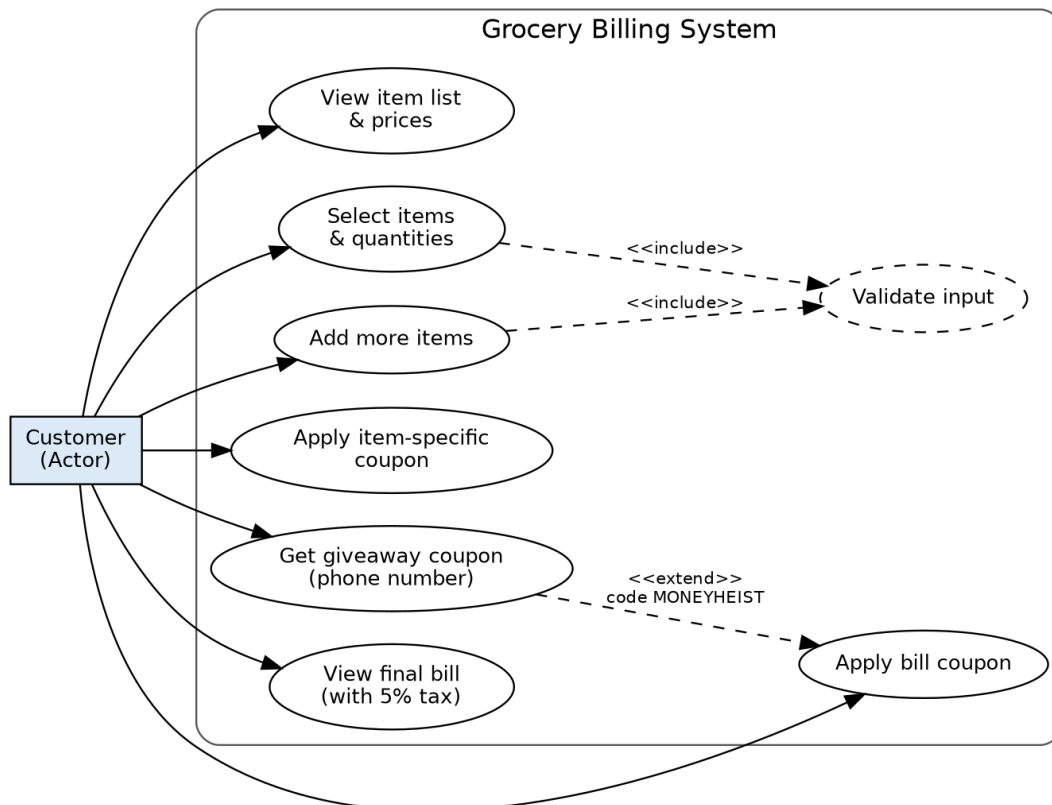


Figure 2: Use case diagram

7.2 Workflow Diagram

The flow below shows the order of steps in one run of the program. The loop after the validity check keeps the user on the input step until the entry is correct.

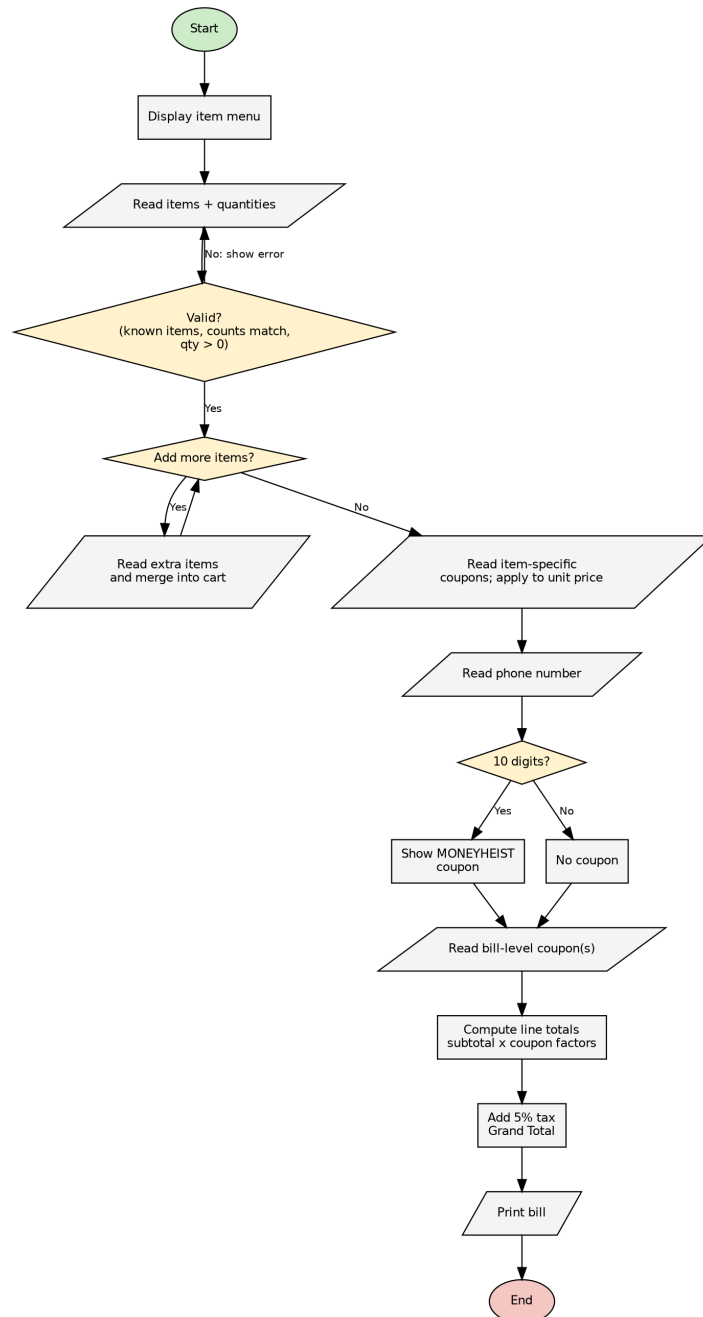


Figure 3: Workflow (flowchart) of a billing session

7.3 Sequence Diagram

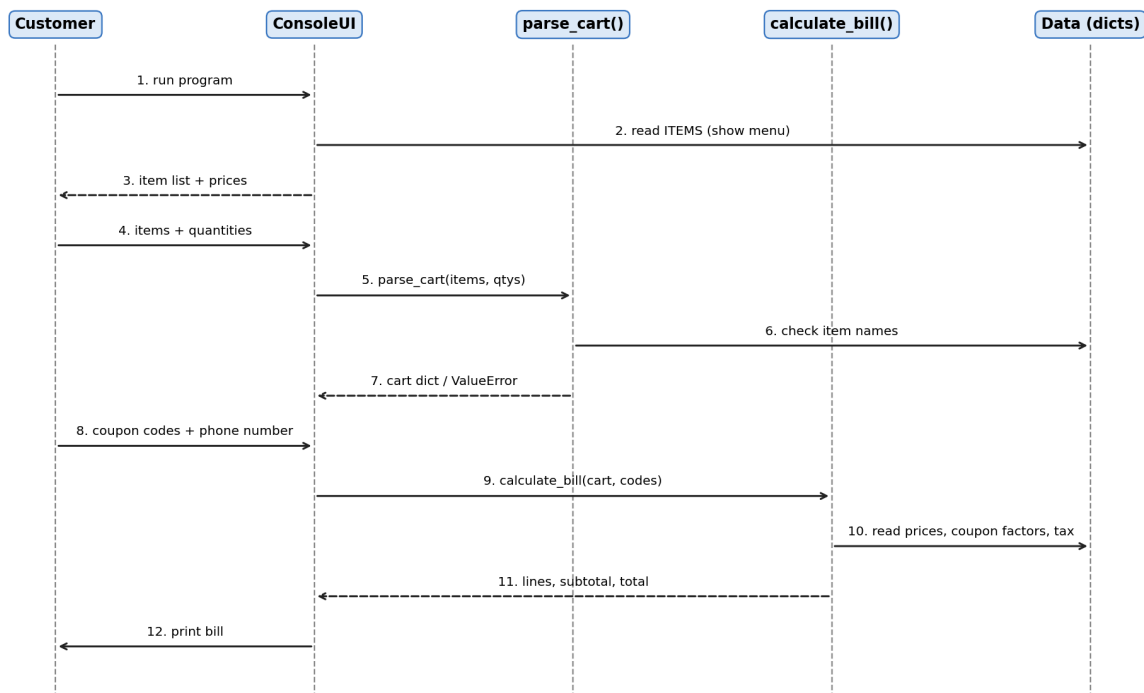


Figure 4: Sequence diagram of a normal billing session

7.4 Class / Component Diagram

The implementation uses functions and dictionaries rather than classes. The diagram shows the logical model: an object-oriented view of the same responsibilities, which is also how the code could be extended in future (see Section 14).

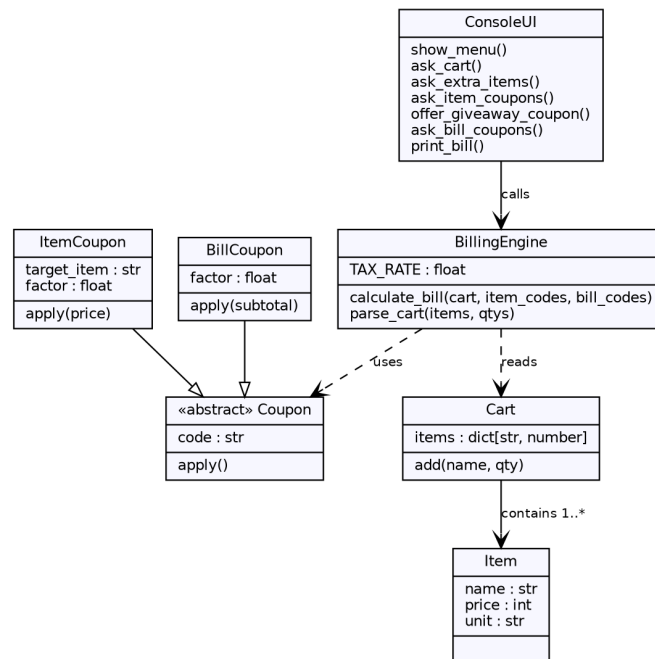


Figure 5: Class / component diagram (logical view)

7.5 ER Diagram

The current program stores nothing permanently, so there is no database. The ER diagram below is the **logical data model** of the information the program handles, and is the schema that would be used if bills and the catalogue were stored in a database in the future.

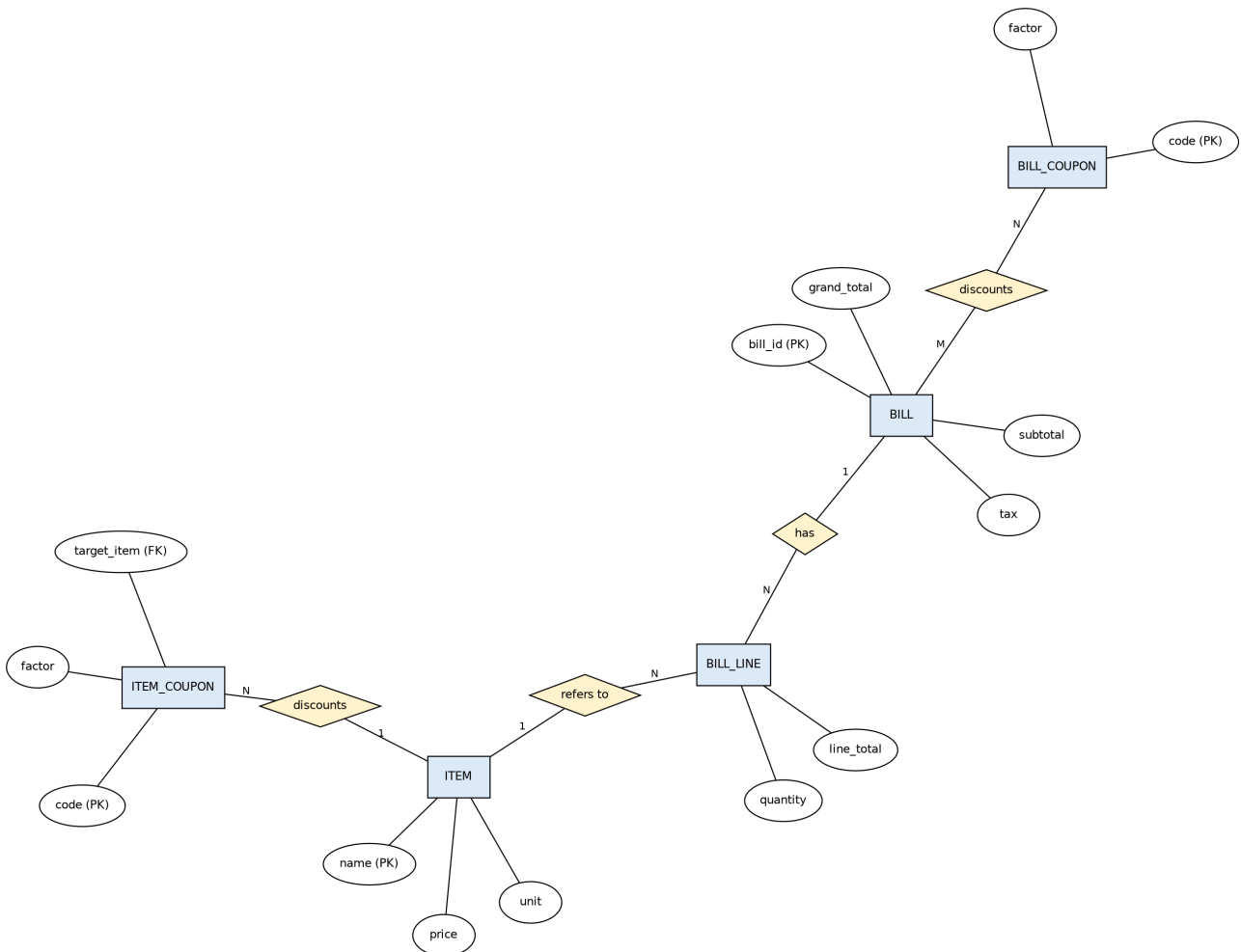


Figure 6: Entity-relationship diagram (logical model)

8. Design Decisions and Rationale

Decision	Rationale
Plain Python instead of NumPy	The original used NumPy arrays for lists of strings and numbers. NumPy adds nothing here, and its arrays broke the program (for example, <code>l1 + l5</code> adds element-wise instead of joining lists). Built-in lists and dicts are simpler and correct.
Dictionaries for prices and coupons	One place to change a price or add a coupon. Removes the long <code>if / elif</code> chain that hard-coded every price a second time.
Separate calculation from input/output	<code>calculate_bill()</code> takes plain data and returns plain data, so it can be unit-tested without a keyboard.
Cart stored as a dict <code>{item: qty}</code>	Repeated items are merged automatically; item names cannot get out of step with quantities as they could with two parallel arrays.
Validation loop on input	A wrong entry asks again instead of crashing or producing a wrong bill.
Item coupons change the unit price; bill coupons change the subtotal	Matches the meaning of the coupon types. Item coupons apply once per code (duplicates are ignored). Bill coupons are applied one after another.
Tax applied after discounts	Tax is charged on the amount the customer actually pays, which is the usual practice.
Coupon codes are case-insensitive	Users may type <i>riseup</i> , <i>RISEUP</i> or <i>RiseUp</i> .
Console interface	Meets the scope of the assignment; the layered design allows a GUI or web interface to reuse the same logic later.

9. Implementation Details

Language and tools. Python 3, standard library only. Unit tests use the built-in `unittest` module. The project consists of two files: `grocery_system.py` (program) and `test_grocery.py` (tests).

9.1 Main functions

Function	Purpose
<code>show_menu()</code>	Prints the numbered catalogue from the <code>ITEMS</code> dictionary.
<code>parse_cart(item_text, qty_text, cart)</code>	Splits both inputs, converts quantities to numbers, checks that counts match, items exist and quantities are positive, then merges into the cart. Raises <code>ValueError</code> with a friendly message otherwise.
<code>ask_cart()</code> , <code>ask_extra_items()</code>	Loop until valid input; the second lets the user add more items.
<code>ask_item_coupons()</code>	Reads <code>N</code> and then <code>N</code> item-specific codes; unknown codes are ignored.
<code>offer_giveaway_coupon()</code>	Accepts only a 10-digit numeric phone number, then shows <code>MONEYHEIST</code> .
<code>ask_bill_coupons()</code>	Reads bill-level codes; unknown codes are ignored with a message.
<code>calculate_bill(cart, item_codes, bill_codes)</code>	Copies base prices, applies item-coupon factors, computes each line and the subtotal, multiplies by each bill-coupon factor, and adds 5% tax.
<code>print_bill()</code>	Prints the aligned bill with rate, amount, tax and grand total.

9.2 Pricing algorithm

Let p_i be the base price of item i , f_i the item-coupon factor (1 if none), q_i the quantity and g_j the bill-coupon factors entered.

$$\text{Subtotal} = (\sum p_i \times f_i \times q_i) \times \prod g_j$$

$$\text{Grand Total} = \text{Subtotal} \times 1.05$$

Worked example (Scenario 1 in Section 10): bread 2 x 20 = 40; butter 1 x 25 (RISEUP halves 50) = 25; chicken 2 x 153 = 306; milk 1 x 60 = 60; chips 3 x 10 = 30. Sum = 461. MONEYHEIST (x 0.8) gives 368.80. Tax at 5% is 18.44, so the grand total is 387.24.

9.3 Code snippet: core calculation

```
def calculate_bill(cart, item_codes=(), bill_codes=()):
    prices = {name: price for name, (price, _) in ITEMS.items()}
    for code in set(c.lower() for c in item_codes):
        if code in ITEM_COUPONS:
            item, factor = ITEM_COUPONS[code]
            prices[item] = prices[item] * factor
    lines, subtotal = [], 0.0
    for item, qty in cart.items():
        line_total = prices[item] * qty
        lines.append((item, qty, prices[item], line_total))
        subtotal += line_total
    for code in bill_codes:
        subtotal *= BILL_COUPONS.get(code.lower(), 1)
    total = subtotal * (1 + TAX_RATE)
    return lines, round(subtotal, 2), round(total, 2)
```

10. Screenshots / Results

The following terminal sessions were produced by running the program with the inputs shown after each prompt.

Scenario 1: normal purchase with added items, an item coupon and a bill coupon

```
>>>>>>> ITEMS AVAILABLE IN STORE <<<<<<<<
1. bread = 20 Rs.
2. butter = 50 Rs.
3. cheese = 80 Rs.
4. gum = 1 Rs.
5. chips = 10 Rs.
6. chicken = 153 Rs. per kg
7. icecream = 20 Rs.
8. toothpaste = 40 Rs.
9. chocolate = 20 Rs.
10. milk = 60 Rs. per liter
>>>>>>> LIST OF ITEMS PURCHASED (space separated) <<<<<<<<<
bread butter chicken
>>>>>>> ENTER QUANTITY OF EACH ITEM (space separated) <<<<<<<<<
2 1 2
Would you like to add more items? (yes/no): yes
Enter the items you want to add: milk chips
Enter the quantity of each item respectively: 1 3
Would you like to add more items? (yes/no): no
>>>>>>> ENTER NUMBER OF ITEM SPECIFIC COUPON CODES <<<<<<<<
1
>>>>>> ENTER ITEM SPECIFIC COUPON CODE <<<<<<
riseup
>>>>>>> TO GET COUPON ENTER YOUR PHONE NUMBER <<<<<<<<<
9876543210
>>>>>>> YOUR COUPON CODE <<<<<<<<<
MONEYHEIST

>>>>>>> ENTER COUPON CODE IF ANY <<<<<<<<<
MONEYHEIST

-----
ITEM          QTY    RATE    AMOUNT
-----
bread          2     20.00    40.00
butter         1     25.00    25.00
chicken        2    153.00   306.00
milk           1     60.00    60.00
chips          3     10.00    30.00
-----
The Total price before tax = 368.80 Rs.
Tax (5%)              = 18.44 Rs.
The Grand Total       = 387.24 Rs.
-----
```

Result: subtotal 368.80 Rs., tax 18.44 Rs., grand total 387.24 Rs. This matches the hand calculation in Section 9.2.

Scenario 2: invalid input, invalid phone number and invalid coupon

```
>>>>>>> ITEMS AVAILABLE IN STORE <<<<<<<<
1. bread = 20 Rs.
2. butter = 50 Rs.
3. cheese = 80 Rs.
4. gum = 1 Rs.
5. chips = 10 Rs.
6. chicken = 153 Rs. per kg
7. icecream = 20 Rs.
8. toothpaste = 40 Rs.
9. chocolate = 20 Rs.
10. milk = 60 Rs. per liter
>>>>>>> LIST OF ITEMS PURCHASED (space separated) <<<<<<<<<
bread jam
>>>>>>> ENTER QUANTITY OF EACH ITEM (space separated) <<<<<<<<<
1 1
! 'jam' is not sold in this store. Please try again.

>>>>>>> LIST OF ITEMS PURCHASED (space separated) <<<<<<<<<
bread
>>>>>>> ENTER QUANTITY OF EACH ITEM (space separated) <<<<<<<<<
1 2
! You entered 1 item(s) but 2 quantity value(s). Please try again.

>>>>>>> LIST OF ITEMS PURCHASED (space separated) <<<<<<<<<
bread cheese
>>>>>>> ENTER QUANTITY OF EACH ITEM (space separated) <<<<<<<<<
2 1
Would you like to add more items? (yes/no): no
>>>>>>> ENTER NUMBER OF ITEM SPECIFIC COUPON CODES <<<<<<<<
0
>>>>>>> TO GET COUPON ENTER YOUR PHONE NUMBER <<<<<<<<
12345
Invalid Number >>> No Coupon For You

>>>>>>> ENTER COUPON CODE IF ANY <<<<<<<<
FAKE money
! 'FAKE' is not a valid coupon, ignored.

-----
ITEM          QTY    RATE    AMOUNT
-----
bread          2     20.00    40.00
cheese         1     80.00    80.00
-----
The Total price before tax = 84.00 Rs.
Tax (5%)              = 4.20 Rs.
The Grand Total              = 88.20 Rs.
-----
```

Result: the unknown item *jam* and the mismatched counts are rejected and re-asked; a 5-digit phone number gives no coupon; the invalid code *FAKE* is ignored while *money* is accepted. Cart value is 40 + 80 = 120 Rs.; MONEY (30% off) gives a subtotal of 84.00 Rs.; tax is 4.20 Rs. and the grand total is 88.20 Rs.

11. Testing Approach

Testing combined **automated unit tests** for the calculation and parsing logic with **manual scenario runs** of the whole console flow. Because the pricing code is separate from input/output, it could be tested directly.

11.1 Unit tests (test_grocery.py) - 10 tests, all passing

#	Test	Input	Expected	Result
1	Simple bill	2 bread + 1 milk	Subtotal 100, total 105	Pass
2	Item coupon	2 butter, RISEUP	Subtotal 50	Pass
3	Bill coupon	1 cheese, MONEYHEIST	Subtotal 64	Pass
4	Both coupon types	3 chicken, CLOCKADOODLEDO, FATHER	Subtotal 183.60, total 192.78	Pass
5	Merging duplicates	'bread bread', '1 2'	{bread: 3}	Pass
6	Count mismatch	2 items, 1 quantity	ValueError	Pass
7	Unknown item	jam	ValueError	Pass
8	Negative quantity	bread, -1	ValueError	Pass
9	Non-numeric quantity	bread, 'two'	ValueError	Pass
10	Unknown coupons	gum x10, code 'zzz'	Subtotal 10 (ignored)	Pass

11.2 Manual scenarios

- Normal purchase with add-more, item coupon, phone coupon and bill coupon (Scenario 1).
- Wrong item, wrong quantity count, invalid phone number, invalid coupon (Scenario 2).
- Boundary cases: zero item-coupons entered; empty bill-coupon line; 9- and 11-digit phone numbers; letters in the phone number.

Test command: `python test_grocery.py` (output: *Ran 10 tests ... OK*).

12. Challenges Faced

The original script ran but produced wrong bills. Finding and fixing its problems was the main challenge of the project.

Problem in the first version	Effect	Fix
<code>b.lower == 'add'</code> and <code>k.lower == ...</code> - the parentheses were missing, so a method object was compared with a string	The 'add items' option and every item coupon never worked	Call the method: <code>.lower()</code>
Prices were kept in a 3-element array <code>[10, 50, 153]</code> , but indexes such as <code>l3[6]</code> were used	<code>IndexError</code> , or discounts applied to the wrong items	Single ITEMS dictionary keyed by name
<code>a = a + l3[i]</code> ran for every item before the price x quantity line	One extra unrelated price added to every bill	Removed; the line total is computed once

Problem in the first version	Effect	Fix
<code>l1 = l1 + l5</code> on NumPy arrays	Adds element-wise instead of appending, so added items were lost or raised errors	Cart dictionary and <code>parse_cart()</code> merge
<code>range(0, d+1)</code> for d coupons	Asked for one coupon too many	Loop over <code>range(n)</code>
Coupon codes were added with <code>l4 + w</code> (array arithmetic)	Failed or produced wrong values when lengths differed	Plain list of valid codes
Prices repeated as literals inside a long if/elif chain; butter used <code>l3[2]</code> , bread <code>l3[1]</code>	Wrong price used for bread, butter and chicken	Look up price from the dictionary
No input validation	Crashes on unknown item, letters, or unequal counts	Validation and retry loop
Tax and totals printed without a bill layout	Hard to read	Aligned itemised bill

A second challenge was that the original coupon descriptions were ambiguous, since the code did not make clear which item each coupon belonged to. The mapping used in this report (RISEUP for butter, CLOCKADOODLEDO for chicken, SMOOTHLIKE for cheese) is the most natural reading of the original code and can be changed in one line of the `ITEM_COUPONS` dictionary.

13. Learnings and Key Takeaways

- Choose the right data structure: dictionaries keyed by name are safer than several parallel arrays whose positions must stay in sync.
- Do not add a library (NumPy) unless it solves a real problem; here it introduced bugs.
- Separating calculation from input/output makes the code testable and easier to reason about.
- A missing pair of parentheses (`.lower` versus `.lower()`) is valid Python but silently changes the meaning, which is why tests are important.
- Validate all user input at the boundary and give messages that say how to fix the problem.
- Write the expected result by hand first (worked example) and compare it with the program's output.
- Requirements such as coupon meanings should be written down clearly, because ambiguous rules lead to guesses in the code.

14. Future Enhancements

- Store the catalogue, coupons and bills in a database (SQLite) using the ER model in Section 7.5, with bill history and reports.
- Convert to an object-oriented design (Item, Cart, Coupon, BillingEngine) as in the class diagram.
- Add a graphical interface (Tkinter) or a web interface (Flask / Django) for use on a real counter.
- Stock management: reduce stock after each sale and warn when items run low.
- Coupon rules with expiry dates, usage limits and minimum-purchase conditions; one-time-use codes generated per phone number.
- Support for multiple payment methods, printable / PDF receipts and GST breakdown per item category.
- Product selection by number and barcode scanning instead of typing names.

- Multi-user login for cashier and manager roles.
- Support for fractional weights with unit conversion (grams to kg) for chicken and similar items.

15. References

- Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>
- Python Software Foundation. *unittest - Unit testing framework*. <https://docs.python.org/3/library/unittest.html>
- Python Software Foundation. *Data Structures (lists and dictionaries)*. <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. *PEP 8 - Style Guide for Python Code*. <https://peps.python.org/pep-0008/>
- Chen, P. (1976). The Entity-Relationship Model: Toward a Unified View of Data. *ACM Transactions on Database Systems*, 1(1), 9-36.
- Booch, G., Rumbaugh, J., Jacobson, I. (2005). *The Unified Modeling Language User Guide*, 2nd ed. Addison-Wesley.
- Sommerville, I. (2016). *Software Engineering*, 10th ed. Pearson.
- Graphviz Project. *Graphviz documentation*. <https://graphviz.org/documentation/>